

QUESTION: 15 to increase a spaly tree inc and performe insertion and search operations .A splay tree is a self adjusting search

main.c

Share

Run

Output

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  typedef struct Node {
4      int key;
5      struct Node *left, *right;
6  } Node;
7  Node* newNode(int key) {
8      Node* node = (Node*)malloc(sizeof(Node));
9      node->key = key;
10     node->left = node->right = NULL;
11     return node;
12 }
13 Node* rightRotate(Node* x) {
14     Node* y = x->left;
15     x->left = y->right;
16     y->right = x;
17     return y;
18 }
19 Node* leftRotate(Node* x) {
20     Node* y = x->right;
21     x->right = y->left;
22     y->left = x;
23     return y;
24 }
25 Node* splay(Node* root, int key) {
26     if (root == NULL || root->key == key)
```

Inorder traversal: 20 30 40 50 60
After searching 40, root is: 40

=== Code Execution Successful ===

```
main.c
27     return root;
28     if (key < root->key) {
29         if (root->left == NULL) return root;
30         if (key < root->left->key) {
31             root->left->left = splay(root->left->left, key);
32             root = rightRotate(root);
33         }
34         else if (key > root->left->key) {
35             root->left->right = splay(root->left->right, key);
36             if (root->left->right != NULL)
37                 root->left = leftRotate(root->left);
38         }
39         return (root->left == NULL) ? root : rightRotate(root);
40     }
41     else {
42         if (root->right == NULL) return root
43         if (key < root->right->key) {
44             root->right->left = splay(root->right->left, key);
45             if (root->right->left != NULL)
46                 root->right = rightRotate(root->right);
47         }
48         else if (key > root->right->key) {
49             root->right->right = splay(root->right->right, key);
50             root = leftRotate(root);
51         }
52         return (root->right == NULL) ? root : leftRotate(root);
53     }
54 }
```

main.c

```
53     }
54 }
55 Node* insert(Node* root, int key) {
56     if (root == NULL)
57         return newNode(key);
58     root = splay(root, key);
59     if (root->key == key)
60         return root; // already exists
61     Node* newnode = newNode(key);
62     if (key < root->key) {
63         newnode->right = root;
64         newnode->left = root->left;
65         root->left = NULL;
66     }
67     else {
68         newnode->left = root;
69         newnode->right = root->right;
70         root->right = NULL;
71     }
72     return newnode;
73 }
74 Node* search(Node* root, int key) {
75     return splay(root, key);
76 }
77 void inorder(Node* root) {
78     if (root) {
```

main.c

```
72     return newnode;
73 }
74 Node* search(Node* root, int key) {
75     return splay(root, key);
76 }
77 void inorder(Node* root) {
78     if (root) {
79         inorder(root->left);
80         printf("%d ", root->key);
81         inorder(root->right);
82     }
83 }
84 int main() {
85     Node* root = NULL;
86     root = insert(root, 50);
87     root = insert(root, 30);
88     root = insert(root, 40);
89     root = insert(root, 20);
90     root = insert(root, 60);
91     printf("Inorder traversal: ");
92     inorder(root);
93     printf("\n");
94     root = search(root, 40);
95     printf("After searching 40, root is: %d\n", root->key);
96     return 0;
97 }
```